

How to Stream Agent Responses

What is a Streamed Response?

A streamed response delivers the message content in small, incremental chunks. This approach enhances the user experience by allowing them to view and engage with the message as it unfolds, rather than waiting for the entire response to load. Users can begin processing information immediately, improving the sense of responsiveness and interactivity. As a result, it minimizes delays and keeps users more engaged throughout the communication process.

Streaming References

- [OpenAI Streaming Guide](#)
- [OpenAI Chat Completion Streaming](#)
- [OpenAI Assistant Streaming](#)
- [Azure OpenAI Service REST API](#)

Streaming in Semantic Kernel

[AI services](#) that support streaming in Semantic Kernel use different content types compared to those used for fully-formed messages. These content types are specifically designed to handle the incremental nature of streaming data. The same content types are also utilized within the Agent Framework for similar purposes. This ensures consistency and efficiency across both systems when dealing with streaming information.

Tip

API reference:

- [StreamingChatMessageContent](#)
- [StreamingTextContent](#)
- [StreamingFileReferenceContent](#)
- [StreamingAnnotationContent](#)

Streamed response from ChatCompletionAgent

When invoking a streamed response from a [ChatCompletionAgent](#), the `ChatHistory` in the `AgentThread` is updated after the full response is received. Although the response is streamed incrementally, the history records only the complete message. This ensures that the `ChatHistory` reflects fully formed responses for consistency.

C#

```
// Define agent
ChatCompletionAgent agent = ...;

ChatHistoryAgentThread agentThread = new();

// Create a user message
var message = ChatMessageContent(AuthorRole.User, "<user input>");

// Generate the streamed agent response(s)
await foreach (StreamingChatMessageContent response in
agent.InvokeStreamingAsync(message, agentThread))
{
    // Process streamed response(s)...
}

// It's also possible to read the messages that were added to the
ChatHistoryAgentThread.
await foreach (ChatMessageContent response in agentThread.GetMessagesAsync())
{
    // Process messages...
}
```

Streamed response from OpenAIAssistantAgent

When invoking a streamed response from an [OpenAIAssistantAgent](#), the assistant maintains the conversation state as a remote thread. It is possible to read the messages from the remote thread if required.

C#

```
// Define agent
OpenAIAssistantAgent agent = ...;

// Create a thread for the agent conversation.
OpenAIAssistantAgentThread agentThread = new(assistantClient);

// Create a user message
var message = new ChatMessageContent(AuthorRole.User, "<user input>");

// Generate the streamed agent response(s)
await foreach (StreamingChatMessageContent response in
agent.InvokeStreamingAsync(message, agentThread))
```

```
{
    // Process streamed response(s)...
}

// It's possible to read the messages from the remote thread.
await foreach (ChatMessageContent response in agentThread.GetMessagesAsync())
{
    // Process messages...
}

// Delete the thread when it is no longer needed
await agentThread.DeleteAsync();
```

To create a thread using an existing `Id`, pass it to the constructor of `OpenAIAssistantAgentThread`:

C#

```
// Define agent
OpenAIAssistantAgent agent = ...;

// Create a thread for the agent conversation.
OpenAIAssistantAgentThread agentThread = new(assistantClient, "your-existing-thread-id");

// Create a user message
var message = new ChatMessageContent(AuthorRole.User, "<user input>");

// Generate the streamed agent response(s)
await foreach (StreamingChatMessageContent response in
agent.InvokeStreamingAsync(message, agentThread))
{
    // Process streamed response(s)...
}

// It's possible to read the messages from the remote thread.
await foreach (ChatMessageContent response in agentThread.GetMessagesAsync())
{
    // Process messages...
}

// Delete the thread when it is no longer needed
await agentThread.DeleteAsync();
```

Handling Intermediate Messages with a Streaming Response

The nature of streaming responses allows LLM models to return incremental chunks of text, enabling quicker rendering in a UI or console without waiting for the entire response to complete. Additionally, a caller might want to handle intermediate content, such as results from

function calls. This can be achieved by supplying a callback function when invoking the streaming response. The callback function receives complete messages encapsulated within `ChatMessageContent`.

Callback documentation for the `AzureAIAgent` is coming soon.

Next Steps

[Using templates with agents](#)[Agent orchestration](#)

Last updated on 05/23/2025